F1 RACEOPS

Race Strategy Intelligence & Live Decision Support Platform

Course: CSE494 — Intelligent NoSQL Databases

Project type: Individual project

Technology: Python · PyMongo · MongoDB · Streamlit

Dataset: Original synthetic race telemetry

Project report and implementation guide

Course: CSE494 — Intelligent NoSQL Databases

Project type: Individual project

Dataset: Original synthetic race telemetry generated for this project

Implementation: Python, PyMongo, MongoDB, Streamlit, Pandas, Plotly

1. Abstract

F1 RaceOps is a MongoDB-based application demonstrating how a document database can support an event-oriented race operations workflow. A Python simulator continuously creates synthetic telemetry records and writes them using PyMongo. MongoDB validation rules enforce key data constraints. CRUD functions manage driver records, while aggregation pipelines derive pace summaries, tyre-compound comparisons, pit-stop statistics, incident distributions, and strategy-action counts. A Streamlit dashboard exposes the stored data and calculated summaries. All simulated records are labelled as synthetic; the application does not claim to receive official live Formula 1 telemetry or provide professional-grade strategy advice.

2. Problem statement

Race strategy decisions depend on multiple changing variables: tyre compound and age, lap pace, temperature, pit-stop history, and incidents. This project demonstrates an event-driven data application that stores changing race observations and summarizes them into interpretable operational views. The educational objective is to show why event records, flexible documents, indexes, validation, and aggregation can be useful in a changing telemetry workload.

3. Objectives

1. Design a suitable MongoDB document model for race entities and event data.
2. Enforce required fields, types, enumerations, and numeric ranges with MongoDB validators.
3. Implement Create, Read, Update, and Delete operations through PyMongo.
4. Build aggregation pipelines that return meaningful summaries.
5. Generate telemetry continuously and persist each event in MongoDB.
6. Present race data and analytics in an interactive dashboard.
7. Document assumptions, tests, limitations, and possible extensions.

4. System architecture

Synthetic race simulator → Python event producer → PyMongo → MongoDB → aggregation/strategy functions → Streamlit dashboard

The simulator is the event source. It produces timestamped records with driver, lap, speed, throttle, braking, tyre compound, tyre age, temperature, estimated fuel, lap time, position, and provenance. PyMongo writes telemetry into MongoDB. Separate strategy code evaluates a transparent rule set and stores a recommendation document. The dashboard queries MongoDB and displays summaries.

MongoDB Change Streams are included as an optional extension. They require a replica set; the base project remains usable with standalone MongoDB through regular database queries and dashboard refresh.

5. Dataset and provenance

The seed process creates a reproducible synthetic demo race, driver profiles, sample pit stops, sample incidents, and 1,200 telemetry events by default. The simulator appends more events while running. The data is designed for repeatable testing and database demonstrations, not as factual telemetry or a source of real-world driver comparisons. Every generated telemetry record includes a data_origin field.

6. Document model

Collection	Grain	Purpose
races	One document per race	Circuit, season, lap count, status, conditions
drivers	One document per driver	Driver identity, team, car number
telemetry	One document per observation	Timestamped measurements and race state
pit_stops	One document per pit stop	Stop duration, compounds, positions
incidents	One document per incident	Type, severity, lap, description
strategy_decisions	One document per recommendation	Action, compound suggestion, confidence, rationale

Embedding vs referencing

Telemetry is stored separately from drivers because the event volume can grow without bound. Storing all telemetry in one driver's document would cause unbounded arrays and make time-window queries cumbersome. driver_id and race_id reference stable records without duplicating every driver attribute in each event. A small number of nested attributes could be embedded where they are always read together, but the high-volume event stream remains append-oriented.

7. Validation and indexes

MongoDB JSON Schema validators require essential fields and enforce BSON types, enumerated values, and numeric limits. Examples include a valid telemetry timestamp, speed between 0 and 380 km/h, positive lap number, supported tyre compound, and nonnegative tyre age. Driver and race IDs have unique indexes. Telemetry has compound indexes on race, driver, and timestamp plus a timestamp index to support latest-event and driver-history queries. Validation reduces malformed records but does not prove that simulated values represent physical reality.

8. CRUD operations

• **Create:** insert a driver document with a unique driver_id.

• **Read:** retrieve a driver by identifier and query the latest telemetry events by timestamp.

• **Update:** change a driver's team or update a race's operational status.

• **Delete:** delete a designated demo driver record. The demonstration script uses a dedicated test ID and removes it before and after the test to avoid modifying seeded driver data.

The CRUD implementation is in raceops/crud.py; scripts/demo_crud.py prints the results of each operation.

9. Aggregation pipelines

9.1 Driver pace summary

$match selects one race; $group computes event count, average lap time, average speed, maximum tyre age, and average tyre temperature by driver; $sort ranks average lap time. The result is a descriptive summary of simulated events, not a definitive driver ranking.

9.2 Tyre-compound comparison

Groups telemetry by compound and computes average lap time, tyre age, and temperature. This enables a basic comparison while highlighting the need to control for tyre age, track conditions, traffic, and stint selection before drawing strategic conclusions.

9.3 Pit-stop analysis

Uses $lookup to attach driver/team metadata to pit-stop documents, $unwind to work with matched driver records, $addFields to calculate position delta, and $group to summarize average stop duration and average position delta by team.

9.4 Incident summary

Groups incident documents by type and severity and counts events. This provides a compact operational view of the event log.

9.5 Strategy action counts

Groups recommendations by action (STAY_OUT, PIT_SOON, PIT_NOW, MONITOR) to show how the rules engine is behaving over the current race.

10. Continuous ingestion and strategy rules

The Python simulator creates a new timestamped event, inserts it through PyMongo, and stores a strategy recommendation. Occasional simulated pit stops and safety-car events demonstrate how event types can influence decisions. The initial strategy engine is a transparent rule-based baseline: tyre-age thresholds and high temperature trigger actions; a safety-car event can trigger a pit-now recommendation. The confidence field is a configured demonstration score, not a statistically calibrated probability.

This project demonstrates continuous event generation and database ingestion. It does not use Apache Spark or Kafka because neither is necessary for the assignment-scale workload. A distributed broker/stream processor would be justified only if requirements demanded higher throughput, multiple independent consumers, replay guarantees, or distributed processing.

11. Dashboard

The Streamlit interface displays event, driver, incident, and decision counts; tyre compound summaries; driver pace; latest telemetry; incident distribution; and strategy-action distribution. Start the simulator in a separate terminal to populate fresh events. Refresh the dashboard to retrieve the latest stored state.

12. Testing plan and acceptance criteria

1. Database connection and seeding complete without errors.
2. Six collections exist and expected validators/indexes are installed.
3. CRUD demo shows successful create, read, update, and delete.
4. Aggregation script returns grouped output for available seeded data.
5. Simulator inserts new timestamped telemetry and strategy documents.
6. Dashboard displays counts and summaries when MongoDB is available.
7. Invalid documents are rejected by MongoDB validators (verify using MongoDB shell or Compass).
8. Strategy unit tests cover old soft tyres, safety-car handling, hot tyres, and confidence range.
9. Change Stream listener is tested only against a replica-set deployment; unsupported standalone mode is documented.
10. Missing database produces a clear connection error rather than a false success message.

13. Limitations and ethical presentation

This is a synthetic simulation and does not use official live F1 telemetry. Race dynamics, tyre degradation, position changes, and confidence scores are simplified. A production version would calibrate models using documented public data, account for pit-loss, traffic, undercut/overcut, weather forecasts, safety-car probabilities, and uncertainty, then evaluate strategy recommendations across held-out races. Do not present simulated outputs as actual F1 results.

14. Conclusion

F1 RaceOps demonstrates the required MongoDB concepts in a domain with naturally evolving event data. It combines document modeling, validation, CRUD, PyMongo, aggregation, continuous event generation, and a dashboard. Its most important engineering distinction is separating stable entities from high-volume telemetry and keeping all synthetic data provenance explicit.

15. Viva preparation

Why MongoDB? Telemetry and race events are naturally represented as timestamped documents, and document shapes can evolve as new measurements are introduced. MongoDB supports validation, indexes, aggregation, and time-ordered queries.

Why keep telemetry separate from drivers? Telemetry is high-volume and unbounded; embedding it in driver documents creates large, frequently updated documents and makes retention/query patterns harder to manage.

What is aggregation? A staged data-processing pipeline. This project uses $match, $group, $avg, $max, $sort, $lookup, $unwind, and $addFields.

Is the data live from Formula 1? No. It is generated by our own simulator and labelled synthetic. The simulator demonstrates continuous ingestion, not access to actual car telemetry.

Why not Spark? The assignment's scale does not require a distributed stream-processing engine. Python and PyMongo demonstrate the pipeline with lower setup risk. Spark or Kafka could be a future extension if throughput or distributed processing were required.

What are Change Streams? A MongoDB feature that exposes database changes to consumers. They generally require a replica set or sharded cluster; they are optional here and not required for core functionality.

What is the strategy model? A transparent rule-based baseline, not machine learning and not a validated professional strategy model. The confidence value is illustrative.

How would you improve the project? Calibrate tyre degradation using sourced historical data, add replayable event logs, test strategy outcomes, use replica-set Change Streams, add role-based access, and monitor ingestion latency and failures.